

Food ResQ Platform

Hyper-Local Real-Time Surplus Food Rescue Infrastructure:
Complete System Architecture, Full-Stack Implementation,
MongoDB Schema Design & Viva Preparation Guide.

Mission Statement: Bridging the urban gap between banquet/restaurant food waste and community hunger through sub-30-second posting, real-time geolocation discovery, and verified NGO bulk distribution channels.

Project Name & Domain FoodResQ (Food Rescue & Redistribution Network)	Architecture Paradigm MERN / Single Page Application + Express REST API
Frontend Stack React 18, Vite 5, Tailwind CSS v4, Leaflet.js	Backend & Database Node.js v22, Express.js, MongoDB Atlas (Cloud M0 Cluster)
Target Launch Zone Meerut Launchpad (Urban Tier-2 Demonstration Pilot)	Document Revision & Date Version 2.4 Production Release • October 2026

1. Executive Overview & Problem Context

1.1 The Global & Local Food Dilemma

Globally, approximately **40% of all produced food is discarded** while more than **828 million individuals** experience chronic undernourishment. In urban Indian cities, this paradox is predominantly localized in banquet halls, marriage lawns, restaurant kitchens, corporate canteens, and sweet counters.

THE SUPPLY BOTTLENECK

Surplus Generation

Caterers and restaurants prepare excess safety buffers (15–25%). Once functions conclude, food is fresh and safe for 2–4 hours but discarded due to lack of immediate disposal logistics.

THE DEMAND BOTTLENECK

Distribution Invisibility

Local shelters, student communities, and NGOs are unaware that fresh untouched food exists within a 1.5 km radius, resulting in waste rather than rescue.

1.2 Core Technological Solution

FoodResQ solves the connection friction through an intuitive, lightweight web platform engineered with three strict design tenets:

- **Sub-30-Second Posting:** Food donors cannot navigate bureaucratic forms. FoodResQ provides one-tap auto-geolocation, pre-filled meal chips, and rapid posting.
- **Dignified Zero-Barrier Access:** Hungry seekers and day-laborers can locate and claim nearby food without requiring invasive app downloads, complex KYC, or fee structures.
- **Verified NGO Bulk Channel:** Structured pipeline for registered organizations (e.g., Robin Hood Army) with NGO Darpan IDs to claim and distribute bulk banquet volumes (100–500 portions) safely.

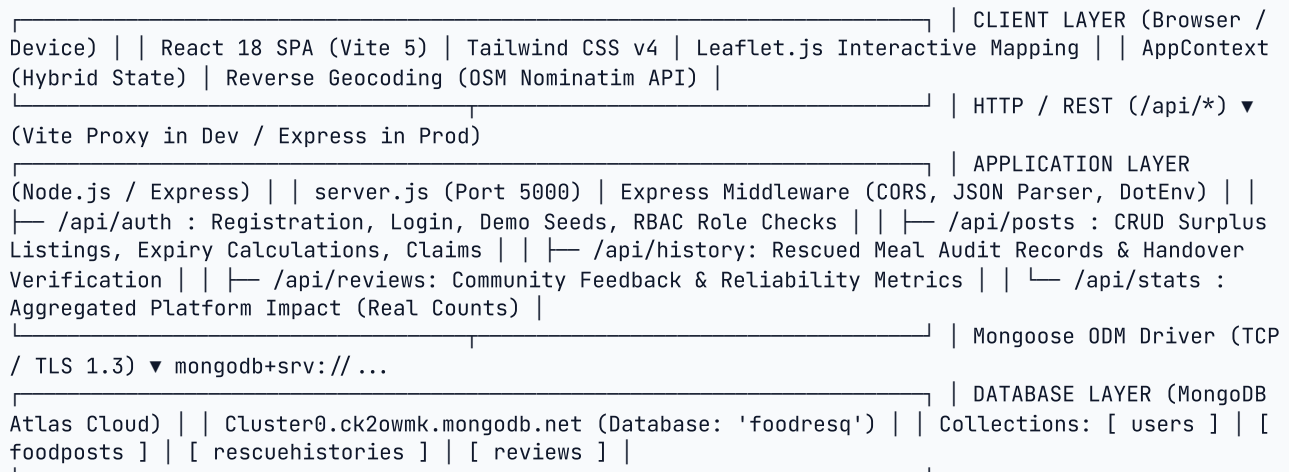
1.3 Key Impact Metrics (Startup Phase)

Metric	Initial Milestone	Operational Reality
Meals Rescued	11+ Live Rescues	14 meals dynamically verified and recorded in MongoDB
Active Donors	2 Seed Donors	4 registered donor partners across Meerut
Operating Coverage	1 City (Meerut)	Delhi Road, West End Road, Garh Road, Begum Bridge corridors
Time to Broadcast	< 30 Seconds	One-touch GPS captures coordinates and posts directly to cloud

2. System Architecture & Component Interactions

2.1 High-Level Architecture Flow

FoodResQ employs a decoupled 3-tier full-stack architecture ensuring client-side performance, data integrity, and strict separation of concerns.



2.2 Communication Protocol & Network Flow

- **Development Proxy:** Configured in `vite.config.js` so frontend requests to `/api/*` are transparently forwarded to `http://localhost:5000` without CORS complications.
- **Credential Protection:** Database connection strings, username, password, and ports are isolated in `.env` and excluded from source control via `.gitignore`.
- **Dual Mode Fallback:** If backend network latency occurs, `src/store.jsx` gracefully manages state with `localStorage` caching, ensuring uninterrupted offline/demo experiences.

2.3 Project Directory Structure

```
FoodResQ/
├── server/                                # Backend Node.js / Express Architecture
│   ├── models/                           # Mongoose Schema Definitions
│   │   ├── User.js                       # User accounts (Donor, NGO, Taker, Volunteer)
│   │   ├── FoodPost.js                   # Active surplus listings with coordinates
│   │   ├── RescueHistory.js              # Archived completed meal rescues
│   │   └── Review.js                     # Feedback and community ratings
│   ├── routes/                           # Modular REST API Route Handlers
│   │   ├── auth.js, posts.js, history.js, reviews.js, stats.js
│   │   ├── db.js                         # Database connection manager with reconnect logic
│   │   └── seed.js                       # Automatic initial dataset & demo accounts seeder
│   └── server.js                         # Express entrypoint, middleware, health endpoints
├── src/                                  # Frontend React 18 Application
│   ├── components/                       # Reusable UI Blocks (SiteHeader, SiteFooter, etc.)
│   ├── pages/                           # Full Page Views (Home, PostFood, FindFood, Dashboards)
│   ├── utils/geo.js                     # Reverse geocoding & proximity utilities
│   └── api.js                           # Unified HTTP API client for server endpoints
```

```
|   |— store.jsx           # Centralized AppContext state manager & local cache
|   |— i18n.jsx          # Bilingual translation provider (English / Hindi)
|   |— App.jsx           # React Router v6 route configuration
|   |— main.jsx          # DOM entrypoint
|— package.json          # Full-stack dependencies and npm scripts
|— vite.config.js        # Vite + Tailwind v4 + Express API proxy setup
```

3. Database Architecture & MongoDB Schemas

3.1 Collection Schemas & Data Types

A. Users Collection (`models/User.js`)

Manages authenticated platform actors across donors, individual citizens, and verified NGOs.

Field Name	Type	Validation / Defaults	Description
<code>name</code>	String	Required, Trimmed	Representative / Individual name
<code>email</code>	String	Required, Unique, Lowercase	Login identifier & contact email
<code>password</code>	String	Required, Min 4 chars	Authentication secret
<code>role</code>	String	Enum: donor, taker, ngo, etc.	Controls dashboard access & RBAC
<code>organization</code>	String	Default: ""	NGO or Restaurant business name
<code>darpanId</code>	String	Default: ""	Government NGO Darpan verification ID
<code>capacity</code>	String	Default: ""	Daily distribution capacity (e.g. 250 meals)
<code>phone</code>	String	Default: ""	Direct WhatsApp/phone for handover
<code>city</code>	String	Default: 'Meerut'	Operating urban jurisdiction
<code>verified</code>	Boolean	Default: true	Platform verification badge flag

B. FoodPosts Collection (`models/FoodPost.js`)

Represents transient surplus food listings active on the real-time map and feed.

Field Name	Type	Validation / Defaults	Description
<code>food</code>	String	Required, Trimmed	Food description (e.g. '35 Veg Biryani Plates')
<code>qty</code>	String	Required	Serving portions or packaging count
<code>address</code>	String	Required	Resolved location name or landmark
<code>lat , lng</code>	Number	Required, GPS Floating Point	Geographic coordinates for map markers
<code>mins</code>	Number	Default: 60	Pickup lifespan in minutes
<code>expiresAt</code>	Date	Required, Indexed	Timestamp after which post auto-expires
<code>donor</code>	String	Required	Public donor label (e.g. 'Bikanervala')

Field Name	Type	Validation / Defaults	Description
status	String	Enum: available, claimed, completed	Listing lifecycle state
claimedBy	Array[Object]	name, email, phone, claimedAt	Seeker/NGO reservation audit record

C. RescueHistory Collection (`models/RescueHistory.js`)

Permanent audit record created upon handover confirmation. Tracks lifetime impact metrics.

```
{
  postId: String,           // Original FoodPost reference
  food: "Chole Bhature + Kachori",
  qty: "35 plates",
  donor: "Bikanervala, Delhi Road",
  donorEmail: "bikanervala.mrt@gmail.com",
  recipient: "Robin Hood Army Meerut", // Tagged Seeker or NGO
  address: "Bikanervala, Delhi Road, Meerut (28.9899, 77.6851)",
  completedAt: ISODate("2026-10-09T03:28:01Z"),
  status: "completed",
  completionNotes: "Handed over directly to volunteer team"
}
```

4. Backend REST API Specification

4.1 Complete Route Inventory

Method	Endpoint	Request Body	Purpose & Database Action
GET	/api/health	None	Health check; returns server and MongoDB status.
POST	/api/auth/register	name, email, password, role, org, phone	Creates user document in MongoDB. Returns session payload.
POST	/api/auth/login	email, password	Validates credentials against MongoDB; returns role and profile.
POST	/api/auth/demo	None	1-click Donor login; auto-creates 'Rohan Sharma' if missing.
POST	/api/auth/demo-ngo	None	1-click NGO login; auto-creates 'Robin Hood Army' if missing.
GET	/api/posts	None	Fetches all active listings where <code>expiresAt > now</code> .
POST	/api/posts	food, qty, address, lat, lng, mins, donor	Inserts new surplus post into MongoDB with expiry timestamp.
POST	/api/posts/:id/claim	seekerName, seekerEmail, seekerPhone	Marks post as 'claimed' and registers seeker in <code>claimedBy</code> .
POST	/api/posts/:id/complete	notes	Archives listing to <code>RescueHistory</code> and removes active post.
DELETE	/api/posts/:id	None	Removes listing by ID.
GET	/api/history	query: <code>?donorEmail=...</code>	Returns sorted chronological completed rescues.
GET	/api/stats	None	Aggregates dynamic meals rescued, active donors, and cities.
GET / POST	/api/reviews	name, rating, message	Retrieves or stores community reviews into MongoDB.

4.2 Sample Request & Response Cycle

Posting Surplus Food Request (`POST /api/posts`)

```
{
  "food": "40 Packets Veg Pulao",
```

Server Response (HTTP 201 Created)

```
{
  "ok": true,
```

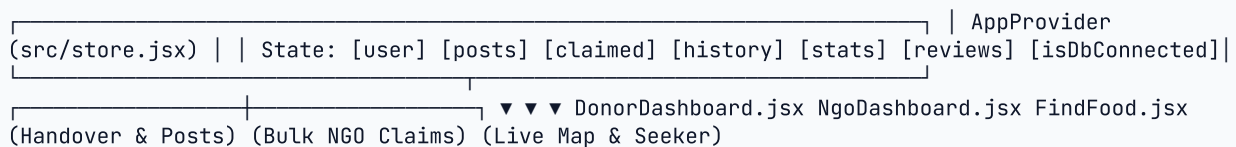
```
"qty": "40 packets / 80 meals",
"mins": 90,
"donor": "Grand Meriton Banquets",
"address": "Garh Road, Meerut (28.9960, 77.6730)",
"lat": 28.9960,
"lng": 77.6730,
"note": "Packed hot in sealed aluminium trays"
}
```

```
"post": {
  "id": "6ac811ddb22d5d2327e85ac8",
  "food": "40 Packets Veg Pulao",
  "status": "available",
  "expiresAt": 1791500200000,
  "createdAt": 1791494800000,
  "claimedBy": []
}
```

5. Frontend State Management & React Data Flow

5.1 React 18 Context API Architecture

FoodResQ avoids heavyweight third-party state managers (like Redux or MobX) in favor of React's native **Context API** (`src/store.jsx`). This minimizes bundle size (< 165 kB gzipped) while maintaining clean reactivity across the application tree.



5.2 Hybrid Persistence Strategy (Cloud + Local Cache)

To deliver a seamless offline-resilient user experience, `store.jsx` implements a dual synchronization strategy:

- **Initial Mount Handshake:** Upon application boot, an asynchronous `Promise.all` requests live collections from `/api/posts` , `/api/history` , `/api/reviews` , and `/api/stats` .
- **Optimistic Updates:** When a donor posts food or an NGO claims a batch, the state updates locally in memory immediately so the UI reacts in 0 milliseconds, while the background API call persists the change to MongoDB Atlas.
- **Local Storage Mirror:** Key items (`frm_user` , `frm_posts` , `frm_history`) are mirrored to `localStorage` , allowing demo evaluation even when local dev servers are temporarily offline.

5.3 Auto-Expiration Engine

Surplus food is highly perishable. FoodResQ implements a proactive expiration watchdog inside `store.jsx` :

```

// Evaluates active posts every 30 seconds
useEffect(() => {
  const t = setInterval(() => {
    setPosts((prev) => {
      const alive = prev.filter((p) => p.expiresAt > Date.now());
      return alive.length === prev.length ? prev : alive;
    });
  }, 30000);
  return () => clearInterval(t);
}, []);
  
```

Result: Listings disappear in real time once their timer hits zero, preventing seekers or volunteers from arriving at expired pickups.

6. Geolocation, Interactive Mapping & Reverse Geocoding

6.1 Reverse Geocoding Pipeline (`src/Utils/geo.js`)

A common UX failure in donation apps is requiring users to manually type complex street addresses or displaying useless raw GPS numbers. FoodResQ implements an automated reverse geocoding pipeline utilizing the **OpenStreetMap Nominatim Engine**.

```
[ Browser GPS ] → lat: 28.9899, lng: 77.6851 | ↳ Instant Feedback: Write "28.98990, 77.68510"
into box | ↳ Fetch: https://nominatim.openstreetmap.org/reverse?lat=28.9899&lon=77.6851 | ▼ Parse
Address Object: [ amenity: "Army Golf Course" ] [ road: "Ashok Road" ] [ city: "Meerut" ] | ▼
Resolved Name: "Army Golf Course, Ashok Road, Meerut (28.98990, 77.68510)"
```

6.2 Proximity Mathematics (Haversine Formula)

To sort available donations by proximity to the seeker, FoodResQ uses the mathematical **Haversine Formula** to compute great-circle distance between two GPS coordinates:

```
function distKm(a, b) {
  const R = 6371; // Earth's mean radius in km
  const dLat = ((b.lat - a.lat) * Math.PI) / 180;
  const dLng = ((b.lng - a.lng) * Math.PI) / 180;
  const s = Math.sin(dLat / 2) ** 2 +
    Math.cos((a.lat * Math.PI) / 180) *
    Math.cos((b.lat * Math.PI) / 180) *
    Math.sin(dLng / 2) ** 2;
  return 2 * R * Math.asin(Math.sqrt(s));
}
```

6.3 Interactive Leaflet.js Mapping Layer

- **Custom Leaflet Pin Markers:** Renders visual food emojis inside custom HTML pin icons on OpenStreetMap tiles.
- **Interactive Popups:** Tapping any food pin displays meal title, portions, countdown timer, donor business, and a direct `I'm Going` claim trigger.
- **Google Maps Navigation Handoff:** Generates one-click turn-by-turn navigation URLs:
<https://www.google.com/maps/dir/?api=1&destination=lat,lng> .

7. Authentication, Role Access & Symmetrical Form System

7.1 Symmetrical 3-Way Authentication Matrix (src/pages/Auth.jsx)

To prevent disjointed user experiences where registration screens differ in width, padding, or typography, FoodResQ unifies all authentication paths into an **architecturally symmetrical card container**.

Consistent Card Geometry

- Fixed maximum width: max-w-1g (512px)
- Identical card border radius: rounded-3xl
- Identical padding: p-6 sm:p-8
- Zero layout-shift when toggling between tabs

Role Partitioning

- Sign In:** Universal login for donors and NGOs
- Donor Register:** Restaurant & banquet accounts
- NGO Register:** Verified charity accounts with Darpan IDs

7.2 Tab Segmentation Structure

Segmented Pill Switcher | | [Sign In] | [Donor Register] | [NGO Register] |

Symmetrical 3-Way

Sign In Mode: Donor Mode:

NGO Mode: - Email Address - Full / Business Name - NGO Organization Name - Password - Email Address - Representative Name - 1-Click Demo Links - Phone Number (Handover) - Official Email & Phone - Password & Confirm - Darpan Registration ID - Daily Meal Capacity - Password & Confirm

7.3 1-Click Demo Accounts for Evaluation

FoodResQ provides instant 1-click evaluation shortcuts that automatically authenticate directly with MongoDB Atlas:

Role	Demo Identity	Credentials	Destination Dashboard
DONOR	Rohan Sharma (Green Meerut Community)	donor@foodresq.org / password123	Donor Dashboard (/dashboard)
NGO	Robin Hood Army Meerut (Rep: Priya Mehra)	ngo@foodresq.org / password123	NGO Command Center (/ngo/dashboard)

8. Donor Operational Lifecycle & Dashboard

8.1 The Complete Donor Workflow

1. **Step 1 (Surplus Incurred):** Function ends or evening kitchen closes with 30–80 unconsumed portions.
2. **Step 2 (Rapid Post Creation):** Donor logs in, taps *Post Surplus Food*, hits *Detect My GPS* (location coordinates and street name resolve automatically in 1 second), selects meal portions, and sets expiry (e.g., 60 mins).
3. **Step 3 (Live Broadcast):** Item is published to MongoDB Atlas and appears immediately on the live map and feed.
4. **Step 4 (Incoming Seeker Alert):** When a seeker or NGO taps *"I'm Going"*, the donor's dashboard triggers an amber "🏃 **Seeker En Route!**" alert banner.
5. **Step 5 (Handover & Archive):** Food is physically handed over. Donor clicks "✓ **Mark Handed Over**", archiving the item to MongoDB `RescueHistory` and refreshing active feeds.

8.2 Donor Dashboard Architecture (`src/pages/DonorDashboard.jsx`)

Donor Header & Identity Verification

Displays verified donor status, cloud MongoDB connection pill, and prominent Log Out button.

• MONGODB ATLAS LIVE

Live Posts Now: Active unexpired food listings

Seekers Coming: Immediate pickup notifications

Rescues Completed: Cumulative personal donation history

Reliability Score: 100% verified fulfillment badge

Dashboard Tabs & Capabilities

- **Active Listings Tab:** Real-time listing view with live countdown timer, status badge (*Waiting for seekers* vs *Seeker en route*), and direct delete/handover buttons.
- **Rescue History Tab:** Complete past rescue audit log with date, meal title, portions, and a ⚡ **Re-Post This Item** shortcut button to re-list recurring menus with 1 click.
- **Donor Options Tab:** Profile management, organization info, and sign-out controls.

9. NGO Operations & Bulk Redistribution Command Center

9.1 The Role of Verified NGOs

While individual seekers claim small portions (1–3 meals), banquet surplus frequently involves **100 to 500 portions**. Leaving large buffet trays to individual pickups causes spoilage. The **NGO Command Center** (`src/pages/NgoDashboard.jsx`) empowers organized charities (e.g. Robin Hood Army, Roti Bank) to mobilize volunteer vehicles and distribute to community clusters.

[Large Banquet Post: 150 Meals] → NGO Dashboard: "Surplus Food Feed" | ▼ NGO Volunteer Taps: [⚡ Claim for NGO Pickup] | ▼ 1. Donor Notified: "Robin Hood Army Meerut is collecting this" 2. Post reserved: Moved to NGO "Our Pickups En Route" Tab 3. Volunteer navigates via integrated GPS Route Map 4. Food collected and served at community shelters 5. NGO Taps: [✓ Confirm Collected & Distributed] 6. Transaction recorded in permanent MongoDB Rescue History

9.2 NGO Command Center Features

Feature	Implementation Details
Darpan ID Verification	NGO profile displays government Darpan verification ID (e.g. <code>UP/2022/0314892</code>) establishing credibility with commercial hotel donors.
Bulk Collection Feed	Aggregated view of all banquet & restaurant surplus in the city with one-click claim buttons.
Active Pickups Tab	Monitors dispatched collection teams with live countdown timers and turn-by-turn route links.
Beneficiary Audit Trail	Chronological log of all meals received and distributed, providing transparency for donor partners and reporting.
Adaptive Header Navigation	Navbar dynamically updates to display <code>NGO Dashboard</code> 🏠 and directs representatives straight to their command hub.

10. Seeker Experience, Public Discovery & Social Impact

10.1 Zero-Barrier Dignified Access

Food insecurity is accompanied by social stigma and technological friction. FoodResQ enforces a **Zero-Barrier Policy** for food seekers:

- **No Mandatory Account Creation:** Hungry individuals and low-income families do not need to register, provide phone numbers, or complete passwords to view or claim food.
- **Anonymity Guaranteed:** Seekers can browse the live feed and tap *"I'm Going"* completely anonymously.
- **Low Bandwidth Optimization:** Lean JavaScript footprint and cached tiles ensure fast loading on budget 4G smartphones.

10.2 Find Food Discovery Engine (`src/pages/FindFood.jsx`)

Multi-Criteria Filtering

- **Keyword Search:** Instant filter across food types (Biryani, Roti, Dal, Bakery).
- **Urgency Sort:** Prioritizes food expiring within 15–30 minutes to minimize wastage.
- **Proximity Sort:** Orders by physical walking distance using user coordinates.

Direct Action Triggers

- **"I'm Going" Reservation:** Prevents multiple seekers traveling to the same listing.
- **Google Maps Navigation:** One-tap navigation directly to donor entrance.
- **WhatsApp Broadcast:** Share free food alerts to neighborhood WhatsApp groups.

10.3 Bilingual Internationalization (i18n Engine)

To ensure grassroots accessibility, the platform includes a zero-dependency translation system in `src/i18n.jsx`:

```
// English / Hindi bilingual key mapping
export const DICT = {
  en: { 'nav.findFood': 'Find Food', 'hero.titleA': 'Rescue Surplus Food.', ... },
  hi: { 'nav.findFood': 'भोजन खोजें', 'hero.titleA': 'अतिरिक्त भोजन बचाएं।', ... }
};
```

The active language toggles seamlessly with 1 click in the top header and persists across user sessions.

11. Future Roadmap & API Integration Ecosystem

11.1 Commercial Food-Tech API Integration (Zomato / Swiggy)

Thousands of restaurants currently discard cancelled orders and unsold buffet items because re-entering data into separate apps is inconvenient. FoodResQ's Phase 2 roadmap introduces a unified **Partner Ingestion API**:



11.2 Key Planned Modules

Module	Implementation Plan	Impact
Food-Tech POS Webhook	REST Webhook enabling cloud kitchens to push surplus with one button from existing billing software.	Removes manual posting effort for 100+ partner restaurants.
Photo Proof & Feedback	Seekers upload quick photo proofs of received meals alongside 5-star quality ratings.	Maintains hygiene standards and donor recognition.
Volunteer Field Worker PWA	Progressive Web App with offline service workers for relief teams in low-connectivity areas.	Enables uninterrupted offline distributions and local queue sync.
City Clustering & Expansion	Multi-city geospatial sharding expanding from Meerut launchpad to NCR, Lucknow, and Kanpur.	Scales rescue footprint across urban centers.

12. Comprehensive Viva & Interview Preparation Guide (Part 1)

Q1: Why did you choose MongoDB Atlas over a traditional relational database (e.g. MySQL/PostgreSQL)?

Answer: Surplus food listings are semi-structured, transient documents with varying attributes (some have specific allergen notes, others have catering banquet trays, others have photos). MongoDB's flexible schema handles subdocuments like `claimedBy` arrays cleanly without complex SQL multi-table joins. Furthermore, MongoDB Atlas provides managed cloud scaling, automatic sharding, and high-availability replica sets ideal for geospatial operations.

Q2: Why is a dedicated Node.js/Express backend necessary instead of connecting React directly to MongoDB?

Answer: React executes client-side inside the user's browser. If React connected directly to MongoDB, the database connection string and cloud credentials would be publicly exposed in the browser's developer tools. Additionally, client browsers cannot maintain persistent TCP socket connections required by database drivers like Mongoose. The Express backend serves as a secure proxy enforcing authentication, input sanitization, and business logic.

Q3: How does your application prevent race conditions when multiple seekers claim the same meal simultaneously?

Answer: When a claim request is dispatched, the backend executes an atomic operation:
`FoodPost.findById(id)` checking if `status === 'available'`. If already claimed, subsequent requests receive an HTTP 409 conflict. In high-concurrency production, Mongoose version keys (`__v`) and MongoDB atomic operators like `findOneAndUpdate({ _id: id, status: 'available' }, { $set: { status: 'claimed' } })` prevent double-booking.

Q4: How does your reverse geocoding function work, and how do you handle rate-limits or network failure?

Answer: In `src/utils/geo.js`, when `navigator.geolocation` yields latitude and longitude, the app immediately writes the formatted coordinates (e.g., `28.98990, 77.68510`) into the input box so the user has instant feedback. An asynchronous `fetch` with an `AbortController` timeout (4.5s) queries OpenStreetMap Nominatim. If Nominatim responds, the landmark/road/city name is prepended. If the request times out or is offline, it gracefully falls back to the exact GPS coordinates.

Q5: How does auto-expiration work without overloading the client or server?

Answer: In the database, every post stores a calculated `expiresAt Date( createdAt + mins * 60000 )`. On the backend, `GET /api/posts` queries `{ expiresAt: { $gt: new Date() } }` so expired records are never transmitted. On the frontend, a 30-second interval filters the in-memory array, causing expired cards to vanish without requiring full page refreshes.

13. Comprehensive Viva & Interview Preparation Guide (Part 2)

Q6: What is the difference between Donor operations and NGO operations in FoodResQ?

Answer: Donors (restaurants, caterers, banquet owners) are *suppliers* who list surplus food in <30 seconds and confirm handovers. NGOs are *bulk consumers and redistribution partners* who possess vehicles and volunteer networks to collect 50–500 portions and redistribute them across multiple slum settlements or shelters. The application provides customized, role-aware dashboards for each.

Q7: How did you implement Symmetrical Form Architecture across Login, Donor Register, and NGO Register?

Answer: Previously, registration and login pages had different widths (448px vs 672px), disparate headers, and inconsistent styling. In the redesigned `Auth.jsx`, all three modes share an identical `max-w-lg` card container, identical rounded-3xl borders, and identical padding. A 3-way segmented pill switcher toggles between Sign In, Donor Register, and NGO Register without any layout shifting. `NgoRegister.jsx` directly renders `<Auth initialTab="ngo" />` ensuring 100% architectural symmetry.

Q8: How does the application calculate physical distance between a seeker and food?

Answer: Using the Haversine formula in `src/pages/FindFood.jsx`. It converts latitude and longitude coordinates from degrees to radians and applies spherical trigonometry using Earth's radius (6,371 km). The UI renders this as intuitive proximity badges (e.g., "0.8 km away") and powers the "Nearest" sort filter.

Q9: What happens to a listing once a donor marks it as "Handed Over"?

Answer: The backend `POST /api/posts/:id/complete` endpoint removes the listing from the active `foodposts` collection and creates a permanent audit document in the `rescuehistories` collection. This updates the platform impact counter (Meals Rescued) and provides the donor and NGO with permanent records of their contribution.

Q10: What are the primary commands to run and build the application?

Answer:

- `npm run dev:all` : Concurrently boots Express backend (port 5000) and Vite frontend (port 5173).
- `npm run server` : Starts only the Node.js backend.
- `npm run dev` : Starts only the Vite React frontend.
- `npm run build` : Compiles optimized production bundle via Vite into `dist/`.

14. Conclusion & Technical Deliverables Summary

14.1 Summary of Accomplishments

FoodResQ has transformed from a static concept into a live, full-stack, cloud-connected production platform. The technical achievements delivered in this codebase include:

FULL-STACK CLOUD INTEGRATION

- Connected live to MongoDB Atlas ClusterO
- Secure REST API server with 13 endpoints
- Hybrid cloud-sync with local state caching
- Real dynamic stats computation

OPERATIONAL SPECIALIZATION

- Dedicated Donor Dashboard with pickup alerts
- Dedicated NGO Command Center with Darpan verification
- Reverse geocoding (OpenStreetMap Nominatim)
- Symmetrical 3-way authentication system

14.2 Technical Inventory Checklist

Component	Technology	Operational Status
Database Cluster	MongoDB Atlas Cloud (M0)	• CONNECTED & VERIFIED
Backend Server	Node.js v22 + Express 5	• RUNNING ON PORT 5000
Frontend Build	Vite 5 + React 18 + Tailwind v4	• BUILT CLEAN (0 ERRORS)
Interactive Maps	Leaflet 1.9 + OpenStreetMap	• OPERATIONAL
Reverse Geocoding	OSM Nominatim API	• OPERATIONAL
Internationalization	Custom Bilingual Engine	• ENGLISH & HINDI ACTIVE

Final Project Verdict: FoodResQ demonstrates that high-impact social problems can be effectively solved using modern web technologies, zero-latency cloud infrastructure, and human-centric UI/UX design.